

Module 6: Sorting Algorithms

การเรียงลำดับข้อมูล: หลักการ ตัวอย่าง โค้ด และประสิทธิภาพ

Selection, Bubble, Insertion, Shell & Merge Sort

การเรียงลำดับข้อมูล (Sorting) คืออะไร?

นิยามของการเรียงลำดับ

กระบวนการจัดเรียงข้อมูลให้อยู่ใน
รูปแบบที่กำหนด นิยมเรียงตามตัวเลข
(Numerical) หรือพจนานุกรม
(Lexicographical) จากน้อยไปมาก

โครงสร้างข้อมูลอาร์เรย์ (Array)

ข้อมูลจำนวน N ตัว จัดเก็บอยู่ใน Array
โดยเริ่มต้นตำแหน่งแรกที่ดัชนี 0
(Index 0) จนถึงดัชนี $N-1$

ตัวอย่างโครงสร้าง Array (ขนาด N)

[0]	[1]	...	[N-2]	[N-1]
34	32	...	21	10

ประเภทของอัลกอริทึมการเรียงลำดับ

$O(n^2)$

1. Basic / Simple Sorts

เหมาะสำหรับข้อมูลขนาดเล็ก อัลกอริทึมเข้าใจง่าย ไม่ซับซ้อน

Bubble Sort

Selection Sort

Insertion Sort

$O(n \log n)$

2. Efficient Sorts

ประสิทธิภาพสูง นิยมนำไปใช้จริงในระบบงานโปรดัคชัน

Merge Sort

Quick Sort

Heap Sort

$O(n+k) / O(n)$

3. Non-Comparison

ประสิทธิภาพสูง ไม่อาศัยการเปรียบเทียบ ใช้คุณลักษณะการกระจายตัวของข้อมูล

Radix Sort

Bucket Sort

สิ่งที่ต้องพิจารณาในการเลือกใช้

1

ความซับซ้อนของเวลา (Time Complexity)

วัดประสิทธิภาพในการทำงานเมื่อจำนวนข้อมูล (N) เพิ่มขึ้นอย่างต่อเนื่อง

2

กรณีแย่มากที่สุด (Worst Case Performance)

พิจารณากรณีที่ใช้เวลามากที่สุด เช่น Merge Sort $O(N \log N)$ แต่ Quick Sort $O(N^2)$

3

ความซับซ้อนของพื้นที่ (Space Complexity)

ปริมาณหน่วยความจำเพิ่มเติมที่ต้องใช้ออกเหนือจากพื้นที่เก็บข้อมูลเริ่มต้น

SELECTION SORT

1. Selection Sort (หลักการทำงาน)

รอบที่ 1 (Index 0 ถึง N-1)

ค้นหาค่าน้อยที่สุด นำไปวางสลับที่ตำแหน่งแรก (Index 0)

รอบที่ 2 (Index 1 ถึง N-1)

ค้นหาค่าน้อยที่สุดถัดไป นำไปวางสลับที่ตำแหน่งถัดมา (Index 1)

การทำงานซ้ำ

ทำซ้ำกระบวนการคัดเลือกค่าต่ำสุดจากส่วนที่ยังไม่เรียงจนกระทั่งหมดข้อมูล

Input [34, 32, 51, 64, 82, 10]

Pass 1 [**10**, 32, 51, 64, 82, 34]

Pass 2 [10, **32**, 51, 64, 82, 34]

Pass 3 [10, 32, **34**, 64, 82, 51]

Pass 4 [10, 32, 34, **51**, 82, 64]

Final 5 [10, 32, 34, 51, 64, 82]

SELECTION SORT

Selection Sort — การวิเคราะห์เวลา

จำนวนการเปรียบเทียบ

$$(N-1) + (N-2) + \dots + 1$$

$$= N(N-1)/2 = O(N^2)$$

คุณสมบัติ

ทุกกรณีใช้จำนวนการเปรียบเทียบเท่ากัน
สลับตำแหน่งจริงได้ไม่เกิน $N-1$ ครั้ง
ใช้พื้นที่เพิ่มเติม $O(1)$; แบบสลับทั่วไปไม่ stable

สำหรับ $N = 6$ เปรียบเทียบ $5 + 4 + 3 + 2 + 1 = 15$ ครั้ง

นับเฉพาะการเปรียบเทียบค่าข้อมูล; ไม่รวมเงื่อนไขควบคุมลูป

2. Bubble Sort (หลักการทำงาน)

การเปรียบเทียบทีละคู่

เปรียบเทียบคู่ติดกันจากซ้ายไปขวา สลับเมื่อ
ค่าซ้ายมากกว่าค่าขวา

ผลของแต่ละรอบ

ค่ามากที่สุดในส่วนที่ยังไม่เรียงจะเลื่อนไปท้าย
ช่วง รอบถัดไปจึงลดช่วงเปรียบเทียบลง

BUBBLE SORT

Bubble Sort (N = 6) — Phase 1

เริ่มจากข้อมูล Input เปรียบเทียบจากซ้ายไปขวา

ก่อนเริ่ม [34, 32, 51, 64, 82, 10]

คู่ที่	เปรียบเทียบ	การทำงาน	ข้อมูลหลังเปรียบเทียบ					
1	34 > 32	สลับ	32	34	51	64	82	10
2	34 > 51	ไม่สลับ	32	34	51	64	82	10
3	51 > 64	ไม่สลับ	32	34	51	64	82	10
4	64 > 82	ไม่สลับ	32	34	51	64	82	10
5	82 > 10	สลับ	32	34	51	64	10	82

ผล Phase 1

[32, 34, 51, 64, 10, 82]

82 ซึ่งเป็นค่ามากที่สุด เลื่อนไปอยู่ตำแหน่งสุดท้าย

BUBBLE SORT

Bubble Sort (N = 6) — Phase 2

เริ่มจากผลลัพธ์ของ Phase 1 เปรียบเทียบจากซ้ายไปขวา

ก่อนเริ่ม [32, 34, 51, 64, 10, 82]

คู่ที่	เปรียบเทียบ	การทำงาน	ข้อมูลหลังเปรียบเทียบ					
1	32 < 34	ไม่สลับ	32	34	51	64	10	82
2	34 < 51	ไม่สลับ	32	34	51	64	10	82
3	51 < 64	ไม่สลับ	32	34	51	64	10	82
4	64 > 10	สลับ	32	34	51	10	64	82

ผล Phase 2

[32, 34, 51, 10, 64, 82]

64 อยู่ในตำแหน่งที่ต้องเพิ่มอีก 1 ตัว ส่วน 82 อยู่ถูกตำแหน่งตั้งแต่ Phase 1

BUBBLE SORT

Bubble Sort (N = 6) — Phase 3

เริ่มจากผลลัพธ์ของ Phase 2 เปรียบเทียบจากซ้ายไปขวา

ก่อนเริ่ม [32, 34, 51, 10, 64, 82]

คู่ที่	เปรียบเทียบ	การทำงาน	ข้อมูลหลังเปรียบเทียบ					
1	32 < 34	ไม่สลับ	32	34	51	10	64	82
2	34 < 51	ไม่สลับ	32	34	51	10	64	82
3	51 > 10	สลับ	32	34	10	51	64	82

ผล Phase 3 [32, 34, 10, 51, 64, 82]

51 อยู่ในตำแหน่งที่ต้องเพิ่มอีก 1 ตัว ส่วน 64 และ 82 อยู่ถูกตำแหน่งแล้ว

BUBBLE SORT

Bubble Sort (N = 6) — Phase 4

เริ่มจากผลลัพธ์ของ Phase 3 เปรียบเทียบจากซ้ายไปขวา

ก่อนเริ่ม [32, 34, 10, 51, 64, 82]

คู่ที่	เปรียบเทียบ	การทำงาน	ข้อมูลหลังเปรียบเทียบ					
1	32 < 34	ไม่สลับ	32	34	10	51	64	82
2	34 > 10	สลับ	32	10	34	51	64	82

ผล Phase 4

[32, 10, 34, 51, 64, 82]

34 อยู่ในตำแหน่งที่ต้องเพิ่มอีก 1 ตัว ส่วน 51, 64 และ 82 อยู่ถูกตำแหน่งแล้ว

BUBBLE SORT

Bubble Sort (N = 6) — Phase 5

เริ่มจากผลลัพธ์ของ Phase 4 เปรียบเทียบจากซ้ายไปขวา

ก่อนเริ่ม [32, 10, 34, 51, 64, 82]

คู่ที่	เปรียบเทียบ	การทำงาน	ข้อมูลหลังเปรียบเทียบ
1	32 > 10	สลับ	10 32 34 51 64 82

ผล Phase 5 [10, 32, 34, 51, 64, 82]

สลับ 32 กับ 10 แล้ว ข้อมูลทั้งหมดเรียงจากน้อยไปมากครบแล้ว

BUBBLE SORT

Bubble Sort — ภาพรวม 5 Phases

Input [34, 32, 51, 64, 82, 10]

Phase 1 [32, 34, 51, 64, 10, 82]

Phase 2 [32, 34, 51, 10, 64, 82]

Phase 3 [32, 34, 10, 51, 64, 82]

Phase 4 [32, 10, 34, 51, 64, 82]

Phase 5 [10, 32, 34, 51, 64, 82]

ไล่จากซ้ายไปขวา ค่ามากที่สุดเลื่อนไปท้ายช่วงในแต่ละ Phase

แบบพื้นฐานเปรียบเทียบ $N(N-1)/2$ ครั้ง; แบบตรวจไม่พบการสลับมี Best Case $O(N)$

3. Insertion Sort (หลักการแทรกข้อมูล)

แนวคิดการแทรก (Insertion)

นำข้อมูลทีละตัวมาวางแทรกในตำแหน่งที่ถูกต้องของส่วนที่เรียงลำดับแล้ว เหมือนกับการเรียงไฟในมือ

การวิเคราะห์ Inversions

จำนวนรอบทำงานสัมพันธ์กับ Inversions (คู่ที่ $i < j$ แต่ $A[i] > A[j]$)
ระยะเวลาดำเนินการ $O(I + N)$

ประสิทธิภาพ Insertion Sort

Worst Case: $O(N^2)$ เมื่อข้อมูลเรียงกลับด้าน

Best Case: $O(N)$ เมื่อข้อมูลเรียงแล้ว

Space: $O(1)$ เป็น In-place sorting

Insertion Sort — ตัวอย่างข้อมูลทั่วไป

Input [34, 8, 64, 51, 32]

ข้อมูลยังไม่เรียง

Pass 1 [8, 34, 64, 51, 32]

แทรก 8 หน้า 34: ขยับ 1 ตัว

Pass 2 [8, 34, 64, 51, 32]

64 อยู่ถูกตำแหน่ง: ไม่ขยับ

Pass 3 [8, 34, 51, 64, 32]

แทรก 51 หน้า 64: ขยับ 1 ตัว

Pass 4 [8, 32, 34, 51, 64]

แทรก 32 หลัง 8: ขยับ 3 ตัว

ตัวอย่างนี้มี 5 inversions และขยับข้อมูล 5 ครั้ง

Average Case เป็นค่าเฉลี่ยจากการกระจายอินพุต; ค่าสำหรับอินพุตสุ่มคือ $\Theta(N^2)$

Insertion Sort — Best Case

Input [10, 20, 30, 40, 50]

ข้อมูลเรียงแล้ว

Pass 1 [10, 20, 30, 40, 50]

$20 > 10$: ไม่ขยับ

Pass 2 [10, 20, 30, 40, 50]

$30 > 20$: ไม่ขยับ

Pass 3 [10, 20, 30, 40, 50]

$40 > 30$: ไม่ขยับ

Pass 4 [10, 20, 30, 40, 50]

$50 > 40$: ไม่ขยับ

เปรียบเทียบ $N-1$ ครั้ง และขยับ 0 ครั้ง

Best Case $\mathcal{O}(N)$ เมื่อข้อมูลเรียงจากน้อยไปมากอยู่แล้ว

Insertion Sort — Worst Case

Input [50, 40, 30, 20, 10]

ข้อมูลเรียงจากมากไปน้อย

Pass 1 [40, 50, 30, 20, 10]

แทรก 40 ขยับ 1 ตัว

Pass 2 [30, 40, 50, 20, 10]

แทรก 30 ขยับ 2 ตัว

Pass 3 [20, 30, 40, 50, 10]

แทรก 20 ขยับ 3 ตัว

Pass 4 [10, 20, 30, 40, 50]

แทรก 10 ขยับ 4 ตัว

จำนวนการขยับ = $1 + 2 + \dots + (N - 1) = N(N - 1)/2$

Worst case $\mathcal{O}(N^2)$; สำหรับ $N = 5$ มีการขยับ 10 ครั้ง

Insertion Sort — Inversions

An inversion is an ordered pair of indices (i, j) such that

$i < j$ and $A[i] > A[j]$

Input [34, 8, 64, 51, 32, 21]

9 inversions

(34, 8), (34, 32), (34, 21)

(64, 51), (64, 32), (64, 21)

(51, 32), (51, 21), (32, 21)

Running time

$O(I + N)$ เป็นกรณีเฉพาะของข้อมูลนั้น

I = number of inversions in the original array N = number of elements

INSERTION SORT

Insertion Sort — C++

```
int a[4] = {38, 18, 12, 7};
int p, j, tmp, N = 4;
for (p = 1; p < N; p++) // phase
{
    for (j = p; j > 0; j--)
    {
        if (a[j] < a[j - 1])
        {
            tmp = a[j];
            a[j] = a[j - 1];
            a[j - 1] = tmp;
        }
        else
            break;
    }
}
```

การทำงาน

p เริ่ม Phase ใหม่

j เปรียบเทียบกับ
สมาชิกด้านซ้าย

สลับเมื่อ $a[j] < a[j-1]$

หยุดเมื่อไม่ต้องสลับ

ผลลัพธ์

[7, 12, 18, 38]

$N = 4$ ตรงกับจำนวนสมาชิกในอาร์เรย์; โค้ดนี้ใช้การสลับคู่ติดกัน

4. Shell Sort — Gapped Insertion Sort

นิยาม

Shell Sort ทำงานโดยเรียงลำดับสมาชิกที่อยู่ห่างกันเป็นระยะทางหนึ่งที่เรียกว่า **"Gap"** (หรือ **Increment/Interval**) ก่อน ซึ่งจะแบ่งอาร์เรย์ออกเป็น Sub-lists ย่อยๆ แล้วทำการเรียงลำดับแต่ละ Sub-list ด้วยวิธี Insertion Sort เมื่อเสร็จสิ้นในแต่ละรอบ ค่า Gap จะถูกลดขนาดลงเรื่อยๆ จนกระทั่ง Gap เหลือเท่ากับ 1 ซึ่งเทียบเท่ากับการทำ Insertion Sort ปกติบนข้อมูลที่ **"เกือบเรียงลำดับแล้ว"** (Nearly Sorted Data) ส่งผลให้ความซับซ้อนของเวลาดีขึ้นอย่างมากเมื่อเทียบกับ $O(N^2)$ ของ Insertion Sort เดิม

ความหมายของ h-sorted

ทุกกลุ่มดัชนี $r, r+h, r+2h, \dots$ เรียงลำดับแล้ว
สำหรับ $r = 0, 1, \dots, h-1$

Shell Sort เป็น comparison sort

SHELL SORT

Shell Sort — รอบ gap = 3

Input [34, 32, 51, 64, 82, 10]

กลุ่ม	ดัชนี	ก่อนเรียง	หลังเรียง
0	0, 3	[34, 64]	[34, 64]
1	1, 4	[32, 82]	[32, 82]
2	2, 5	[51, 10]	[10, 51]

ผล gap = 3: [34, 32, 10, 64, 82, 51]

แต่ละกลุ่มเรียงแล้ว แต่ทั้งอาร์เรย์ยังไม่เรียงครบ ต้องทำรอบ gap = 1

SHELL SORT

Shell Sort — รอบ gap = 1

เริ่ม [34, 32, 10, 64, 82, 51]

แทรก 32 [32, 34, 10, 64, 82, 51]

แทรก 10 [10, 32, 34, 64, 82, 51]

แทรก 64 [10, 32, 34, 64, 82, 51]

แทรก 82 [10, 32, 34, 64, 82, 51]

แทรก 51 [10, 32, 34, 51, 64, 82]

ผลสุดท้าย: [10, 32, 34, 51, 64, 82]

gap = 1 คือ Insertion Sort ปกติ บนข้อมูลที่ผ่านการจัดเรียงบางส่วนแล้ว

การเลือก Gap Sequence

ชุด gap	วิธีกำหนด	ตัวอย่าง เมื่อ $N = 16$
Shell / Halving	เริ่ม $N/2$ แล้วหาร 2 ปิดลง	8, 4, 2, 1
Hibbard	$2^k - 1, k > 1$	15, 7, 3, 1
Knuth	$h = 3h + 1$	13, 4, 1
Pratt	$2^a \times 3^b, a, b \geq 0$	12, 9, 8, 6, 4, 3, 2, 1

เลือก gap ที่น้อยกว่า N ใช้จากมากไปน้อย และต้องลงท้ายด้วย 1

ตัวอย่าง Knuth ใช้ทุก $gap < N$; บาง implementation เริ่มจากขนาด gap ต่างกัน

Shell Sort — ประสิทธิภาพ

ชุด gap	ขอบเขต Worst Case
Shell / Halving	$O(N^2)$
Hibbard	$O(N^{3/2})$
Knuth ($3h + 1$)	$O(N^{3/2})$
Pratt	$O(N \log^2 N)$

พื้นที่เพิ่มเติม $O(1)$ เมื่อสร้าง gap ทีละค่า เช่น Halving/Knuth

ความเร็วขึ้นกับชุด gap; การเก็บรายการ gap ทั้งชุดต้องใช้พื้นที่เพิ่มตามจำนวน gap

Shell Sort — ประสิทธิภาพ ตย. N=10

ชุด gap	ขอบเขต Worst Case	ลำดับ gap ที่ใช้
Shell / Halving	$O(N^2) = 100$	5, 2, 1
Hibbard	$O(N^{3/2}) = 31.62$	7, 3, 1
Knuth ($3h + 1$)	$O(N^{3/2}) = 31.62$	4, 1
Pratt	$O(N \log^2 N) = 110.35$	9, 8, 6, 4, 3, 2, 1

ความเร็วขึ้นกับชุด gap; การเก็บรายการ gap ทั้งชุดต้องใช้พื้นที่เพิ่มตามจำนวน gap

Shell Sort — C++ (Halving Gaps)

```
void shellSort(int a[], int N)
{
    for (int gap = N / 2; gap > 0; gap /= 2)
    {
        for (int i = gap; i < N; ++i)
        {
            int temp = a[i], j = i;
            while (j >= gap && a[j - gap] > temp)
            {
                a[j] = a[j - gap];
                j -= gap;
            }
            a[j] = temp;
        }
    }
}
```

สำหรับ $N = 6$ โค้ดนี้ใช้ $gap = 3$ แล้ว $gap = 1$

ต่างจาก Insertion Sort ที่ยัดครั้งละ 1: รอบนี้ยัดครั้งละ gap ตำแหน่ง

5. Radix Sort: นิยาม

นิยาม

การเรียงลำดับโดยแยกพิจารณาที่ยี่ทีละหลัก
จัดข้อมูลเป็นกลุ่มตามค่าของหลักนั้น แล้วทำซ้ำจนครบ

LSD Radix Sort ในตัวอย่างนี้

เริ่มจากหลักหน่วย ตามด้วยหลักสิบ และหลักร้อย
แต่ละรอบรักษาลำดับเดิมของข้อมูลที่มีเลขหลักนั้นเท่ากัน

LSD = Least Significant Digit: เริ่มจากหลักที่มีค่าน้อยที่สุด

เป็น non-comparison sort โดยจัดกลุ่มตามหลักของคีย์

ขั้นตอนของ LSD Radix Sort

ขั้นที่ 1

กำหนดฐาน b และจำนวนหลัก d

ตัวอย่างฐานสิบ: $b = 10$ จึงมี bucket p_0 ถึง p_9

ขั้นที่ 2

อ่านข้อมูลจากซ้ายไปขวา

นำแต่ละค่าไปต่อท้าย bucket ตามเลขในหลักที่กำลังพิจารณา

ขั้นที่ 3

รวบรวมจาก p_0 ถึง p_9

รักษาลำดับภายใน bucket ตามลำดับที่นำข้อมูลเข้า

ขั้นที่ 4

เลื่อนไปหลักถัดไปจนทำครบ d รอบ

หลักหน่วย แล้วหลักสิบ แล้วหลักร้อย สำหรับข้อมูลชุดนี้

แต่ละรอบต้อง stable เพื่อรักษาผลการเรียงจากรอบก่อน

ตัวอย่างนี้ใช้จำนวนเต็มไม่ติดลบ และเรียงจากน้อยไปมาก

ข้อมูลตัวอย่างและจำนวนรอบ

Input

64, 8, 216, 512, 27, 729, 0, 1, 343, 125

$N = 10$ $b = 10$ ค่ามากที่สุด = 729

มองจำนวนหลักน้อยกว่าเป็นเลขที่เติม 0 ด้านหน้า

$64 = 064$, $8 = 008$, $0 = 000$, $1 = 001$

$d = 3$ จึงเรียงครบด้วย 3 รอบ: หน่วย สิบ และร้อย

การเติม 0 ใช้เพื่ออธิบายหลักของตัวเลข ค่าข้อมูลยังเท่าเดิม

การหาเลขในแต่ละหลัก

หลัก	สูตร (จำนวนเต็ม)	ตัวอย่าง $n = 729$
หน่วย	$n \% 10$	9
สิบ	$(n / 10) \% 10$	2
ร้อย	$(n / 100) \% 10$	7
พัน	$(n / 1000) \% 10$	0

สูตรทั่วไป: $\text{digit} = (n / \text{exp}) \% b$

ใช้การหารจำนวนเต็ม สำหรับฐานสิบ $\text{exp} = 1, 10, 100, 1000, \dots$

Radix Sort Phase 1: หลักหน่วย

Input 64, 8, 216, 512, 27, 729, 0, 1, 343, 125

Bucket / เลขหลัก	ข้อมูลใน bucket ตามลำดับเข้า
p0 / 0	0
p1 / 1	1
p2 / 2	512
p3 / 3	343
p4 / 4	64
p5 / 5	125
p6 / 6	216
p7 / 7	27
p8 / 8	8
p9 / 9	729

0, 1, 512, 343, 64, 125, 216, 27, 8, 729

รวบรวม bucket จาก p0 ถึง p9 โดยรักษาลำดับภายในแต่ละ bucket

Radix Sort Phase 2: หลักสิบ

Input 0, 1, 512, 343, 64, 125, 216, 27, 8, 729

Bucket / เลขในหลัก	ข้อมูลใน bucket ตามลำดับเข้า
p0 / 0	0, 1, 8
p1 / 1	512, 216
p2 / 2	125, 27, 729
p3 / 3	—
p4 / 4	343
p5 / 5	—
p6 / 6	64
p7 / 7	—
p8 / 8	—
p9 / 9	—

0, 1, 8, 512, 216, 125, 27, 729, 343, 64

รวบรวม bucket จาก p0 ถึง p9 โดยรักษาลำดับภายในแต่ละ bucket

Radix Sort Phase 3: หลักร้อย

Input 0, 1, 8, 512, 216, 125, 27, 729, 343, 64

Bucket / เลขในหลัก	ข้อมูลใน bucket ตามลำดับเข้า
p0 / 0	0, 1, 8, 27, 64
p1 / 1	125
p2 / 2	216
p3 / 3	343
p4 / 4	—
p5 / 5	512
p6 / 6	—
p7 / 7	729
p8 / 8	—
p9 / 9	—

0, 1, 8, 27, 64, 125, 216, 343, 512, 729

รวบรวม bucket จาก p0 ถึง p9 โดยรักษาลำดับภายในแต่ละ bucket

ผลลัพธ์ต่อเนื่องทั้ง 3 Phases

Input 64, 8, 216, 512, 27, 729, 0, 1, 343, 125

ข้อมูลชุดเดิมตลอดตัวอย่าง

Phase 1 0, 1, 512, 343, 64, 125, 216, 27, 8, 729

เรียงตามหลักหน่วย

Phase 2 0, 1, 8, 512, 216, 125, 27, 729, 343, 64

เรียงตามสองหลักท้าย

Phase 3 0, 1, 8, 27, 64, 125, 216, 343, 512, 729

เรียงครบทุกหลัก

ผลสุดท้าย: 0, 1, 8, 27, 64, 125, 216, 343, 512, 729

เมื่อทำครบทุกหลัก ข้อมูลจะเรียงจากน้อยไปมาก

ประสิทธิภาพของ LSD Radix Sort

แบบใช้ Stable Counting Sort ในแต่ละรอบ

N = จำนวนข้อมูล b = ฐาน / จำนวน bucket
 d = จำนวนหลักที่ต้องประมวลผล

เวลาแต่ละรอบ $\mathcal{O}(N + b)$

เวลารวม $\mathcal{O}(d(N + b))$

พื้นที่เพิ่มเติม $\mathcal{O}(N + b)$

เมื่อ b และ d คงที่ เวลารวมเป็น $\mathcal{O}(N)$

ค่าติดลบหรือคีย์รูปแบบอื่นต้องปรับวิธีแทนคีย์และการจัดกลุ่ม